NODEBASE · REFERENCE

BullMQ Worker & Queue Flow

How jobs are produced, durably stored in Redis, atomically claimed by a worker, retried on failure, and finalized — plus the two worker processes this project runs. Redis key states shown throughout were captured live from an isolated probe queue, not inferred from documentation.

BullMQ 6.3.2 ioredis 6 Upstash Redis nodemailer 9 Inngest 3

At a glance

Producer web request → **Redis** wait list → **Claim** wait → active → **Processor** worker.ts → **Outcome** completed / failed

The one idea that makes the rest obvious

Enqueueing and processing happen in **different processes**, connected only by Redis. The web request writes a job and returns; a separate long-lived worker reads it. Everything below is a consequence of that split: durability, at-least-once delivery, retries, and the fact that **a successful HTTP response says nothing about whether the work was done**.

The two workers in this project

QUEUE	WORKER FILE	CONC.	RETRY POLICY	WORK PERFORMED
`email`	`src/queue/worker.ts`	2	`3 attempts` `exp 2000ms`	Sends an OTP email via nodemailer. Triggered by the auth flow.
`cron`	`src/queue/cron-worker.ts`	5	`3 attempts` `exp 5000ms`	Fires a scheduled workflow by sending `workflows/execute.workflow` to Inngest.

Both are launched from mprocs (or directly via the `worker` / `cron-worker` npm scripts), never by the Next.js server.

PHASE 1 Connection

1. `src/lib/queue/connection.ts:18` — `getQueueConnection()`

```
const createQueueConnection = () => {
  const url = process.env.UPSTASH_REDIS_URL;
  if (!url) return null;
  return new Redis(url, {
    maxRetriesPerRequest: null,
    enableReadyCheck: false,
  });
};

let shared: Redis | null | undefined;

export const getQueueConnection = () => {
  if (shared === undefined) shared = createQueueConnection();
  return shared;
};
```

Three decisions here are load-bearing:

• **Memoised in a module-level variable.** Repeated calls reuse one client instead of opening a new connection per request. The `undefined` sentinel distinguishes "not yet resolved" from a legitimately `null` result, so a missing env var is not re-attempted on every call.

• **Returns `null` rather than throwing** when `UPSTASH_REDIS_URL` is absent, letting callers choose their own degradation strategy.

• **Both Redis options are mandatory for BullMQ**, not stylistic:

> **`maxRetriesPerRequest: null` is a hard requirement**
>
> BullMQ uses blocking commands to wait for jobs. If the client is configured to give up on a command, the worker's blocking pop times out and the framework aborts startup. Passing a finite value produces, verbatim:
>
> ```
> BullMQ: Your redis options maxRetriesPerRequest must be null.
> ```
>
> This was confirmed empirically while writing this document — the project code is already correct, and any future refactor of this file must preserve it.

2. `enableReadyCheck: false` matters because Upstash is a serverless offering with no replication and a cold start path; the readiness handshake adds latency without a useful signal. The `UPSTASH_REDIS_URL` value is the **ioredis-compatible** endpoint from the Upstash console (Connection Details → IOREDIS), not the REST URL used elsewhere in the codebase for rate limiting.

PHASE 2 Queue construction (producer side)

---

3 `src/lib/queue/email-queue.ts:7` — `getEmailQueue()`

Mirrors the connection pattern: memoised in `queue`, `null` when Redis is unavailable. The job options define the entire retry contract:

```
new Queue<OtpEmailData>("email", {
  connection,
  defaultJobOptions: {
    attempts: 3,
    backoff: { type: "exponential", delay: 2000 },
    removeOnComplete: true,
    removeOnFail: 1000,
  },
});
```

OPTION	MEANING
`attempts: 3`	Total tries, *not* three retries. One initial attempt plus two retries.
`exponential / 2000`	Delays grow: ~2s, then ~4s. Total elapsed before giving up is roughly 6s.
`removeOnComplete: true`	Deletes the job record the moment it succeeds — nothing is retained.
`removeOnFail: 1000`	Retains the **most recent 1000** failures for inspection, discarding older ones.

`src/lib/queue/cron.ts:25` builds the `cron` queue with the same shape but a 5000ms backoff and `removeOnFail: 100`.

4 `email-queue.ts:29` — `enqueueOtpEmail`

```
export const enqueueOtpEmail = async (data: OtpEmailData) => {
  const emailQueue = getEmailQueue();
  if (!emailQueue) { await sendOtpEmail(data); return; }   // inline fallback
  await emailQueue.add("otp", data);
};
```

This single function is the fork between the two deployment modes. With Redis configured the job is written and the function returns; without it, the email is sent on the request thread. The web server therefore has **no visibility into delivery outcome** in queue mode — `add()` resolving means "durably stored", nothing more.

PHASE 3 What a job looks like in Redis

Captured live by enqueuing one job on an isolated probe queue with no worker running:

STAGE 1 — ENQUEUED, NOTHING CONSUMING

```
bull:__probe_demo:1        [hash,  fields=6]   // the job record
bull:__probe_demo:events   [stream]             // event log, maxLenEvents 10000
bull:__probe_demo:id       [string, val=1]      // auto-increment id counter
bull:__probe_demo:marker   [zset,  card=1]
bull:__probe_demo:meta     [hash,  fields=2]    // queue-level metadata
bull:__probe_demo:wait     [list,  len=1]       // ← THE JOB IS HERE
```

KEY	TYPE	ROLE
`<prefix>:wait`	list	**The pending FIFO.** Job IDs are pushed here atomically at enqueue time. This list *is* the backlog.
`<prefix>:<id>`	hash	Full job state: name, data, timestamps, returnvalue, failedReason, attempt counters.
`<prefix>:id`	string	Monotonic counter behind `INCR`, giving collision-free job IDs.
`<prefix>:events`	stream	Append-only diagnostic log, capped by `opts.maxLenEvents`.
`<prefix>:meta`	hash	Queue-level config, observed containing `version` and `opts.maxLenEvents`.
`<prefix>:marker`	zset	Marker set used by BullMQ's internal bookkeeping and trimming.

The prefix is `bull:<queueName>`, so both project queues are namespaced independently: `bull:email:*` and `bull:cron:*`. Workers can never consume each other's jobs.

PHASE 4

Worker startup

5 `src/queue/worker.ts:5-13`

```
const connection = getQueueConnection();
if (!connection) {
  console.warn("[email-worker] UPSTASH_REDIS_URL is not set — cannot start. " + …);
  process.exit(1);
}
```

The worker **refuses to run degraded**, unlike the producer which falls back to inline sending. The asymmetry is deliberate and correct: a producer can degrade safely, but a consumer with no Redis would spin silently, appearing healthy while processing nothing.

6 `src/queue/worker.ts:15`

```
const worker = new Worker<OtpEmailData>(
  "email",
  async (job) => {
    if (job.name === "otp") await sendOtpEmail(job.data);
  },
  { connection, concurrency: 2 },
);
```

Constructing the `Worker` is what starts the consumption loop — there is no `start()` call. Two background activities begin immediately:

• a **blocking pop loop** that claims jobs as they arrive, and
• the **stalled-job checker**, whose presence is directly observable as the `stalled-check` key appearing in the next capture.

The `connection` passed here is the same shared client, and BullMQ will duplicate it internally for its blocking connection — which is why `maxRetriesPerRequest: null` is mandatory.

The processor guards on `job.name === "otp"` rather than assuming, so future job types on this queue are ignored until handled deliberately.

7 `src/queue/cron-worker.ts:16-40` — the same pattern with a different processor:

```
const worker = new Worker<CronJobData>(
  "cron",
  async (job) => {
    const { workflowId, nodeId, scheduleId } = job.data;
    await inngest.send({
      name: "workflows/execute.workflow",
      data: { workflowId, initialData: { cron: { nodeId, scheduleId,
              timezone: job.data.timezone,
              timestamp: new Date().toISOString() } } },
    });
  },
  { connection, concurrency: 5 },
);
```

> **The cron worker is a dispatcher, not an executor**
> It performs no workflow logic itself. It receives a scheduled tick and hands off to Inngest, which runs `executeWorkflow` . A scheduled workflow therefore depends on **three** independent services being healthy: this worker, the Inngest dev server or cloud, and the Inngest function's own registration. A cron trigger failing silently is often a failure in the second or third, not the first.

PHASE 5 How a job is taken

This is the heart of the system. A worker must claim a job in a way that guarantees **exactly one** worker gets it, even with several workers running against the same Redis.

8 **The claim is a single atomic operation.** BullMQ moves the job ID from the `wait` list to the `active` list as one indivisible Redis command (the `BRPOPLPUSH` / `BLMOVE` family), then writes a lock key. Atomicity is the entire point: a naive `LPOP wait` followed by `LPUSH active` would leave a window in which a crash between the two loses the job permanently, and in which two workers could both read the same ID.

Observed live, mid-processing, with the processor deliberately blocked for 3 seconds:

STAGE 2 — DURING PROCESSING

```
bull:__probe_demo:1            [hash,   fields=8]   // 6 → 8 fields
bull:__probe_demo:1:lock       [string, val=9951c339-…-1da3936a0054:0]
bull:__probe_demo:active       [list,   len=1]      // ← JOB IS HERE NOW
bull:__probe_demo:events       [stream]
bull:__probe_demo:id           [string, val=1]
bull:__probe_demo:marker       [zset,   card=1]
bull:__probe_demo:meta         [hash,   fields=2]
bull:__probe_demo:stalled-check [string, val=1790508344994]   // NEW

// note: the "wait" key is absent — Redis deletes a list when it empties
```

OBSERVATION	WHAT IT PROVES
`wait` gone, `active` len=1	The handoff really did move the entry, rather than copying it. A job is never simultaneously pending and running.
`<id>:lock` appeared	BullMQ writes a token keyed to this attempt (`<uuid>:0`). It is the claim record — only the holder may finalize the job.
`<id>` 6 → 8 fields	In-flight metadata written at claim time (`processedOn` , attempt bookkeeping) without touching `wait` .
`stalled-check` appeared	The watchdog armed. It periodically re-verifies that jobs in `active` are genuinely progressing.

Why `active` and the lock matter operationally

A job appearing in `active` and never leaving means the worker holding it died mid-flight. The stalled checker detects the missing heartbeat and re-queues the job — which is what makes delivery **at-least-once** rather than at-most-once. The flip side is that a processor which succeeds at its side effect but crashes before returning can cause the job to run twice. Section “Findings” covers where that matters here.

PHASE 6 Completion

9 When the processor returns normally, BullMQ stores the return value, moves the job ID from `active` to the `completed` sorted set (scored by completion time), and deletes the lock.

STAGE 3 — AFTER COMPLETION

```
bull:__probe_demo:1          [hash, fields=11]    // 8 → 11 fields
bull:__probe_demo:completed  [zset,  card=1]      // ← NEW
bull:__probe_demo:events     [stream]
bull:__probe_demo:id         [string, val=1]
bull:__probe_demo:meta       [hash,  fields=2]
```

```
bull:__probe_demo:stalled-check [string, val=1790508344994]

// "active" and "1:lock" are both GONE
```

```
state        : completed
returnvalue  : {"ok":true}      // captured from the processor's return
attemptsMade : 1
processedOn  : 1790508344995
finishedOn   : 1790508348103
timestamp    : 1790508343134
```

The lifecycle of the six key mutations across a successful job:

KEY	STAGE 1	STAGE 2	STAGE 3	REASON FOR THE CHANGE
`<id> hash fields`	6	8	11	Progressively records in-flight and final state.
`wait`	len 1	gone	gone	Emptied by the atomic claim; Redis drops empty lists.
`active`	—	len 1	gone	Entry removed once finalized.
`<id>:lock`	—	present	gone	Claim released after the outcome is recorded.
`completed`	—	—	card 1	Terminal state for a successful job.
`stalled-check`	—	present	present	Stays armed for the worker's lifetime.

On the real email queue, Stage 3 is invisible

`removeOnComplete: true` (`email-queue.ts:20`) means the job hash, its `completed` entry, and the ID are deleted the instant the email is accepted by SMTP. The successful states above were therefore only visible because the probe queue used `removeOnComplete: false`. In production there is **no record that an email was ever sent** — see finding 1.

PHASE 7 Failure and retry

10 Throwing from the processor marks the job failed. BullMQ retries it according to `attempts` and `backoff`. Reproduced live with the same options the email queue uses:

```
=== adding a job that always throws (attempts:3, exponential backoff 500ms) ===
  [processor] attempt #1 on job 1 (attemptsMade=0)
  [event] failed: job 1 attempt=1 -> 535-5.7.8 Username and Password not accepted
  [processor] attempt #2 on job 1 (attemptsMade=1)
  [event] failed: job 2 attempt=2 -> 535-5.7.8 Username and Password not accepted
  [processor] attempt #3 on job 1 (attemptsMade=2)
  [event] failed: job 1 attempt=3 -> 535-5.7.8 Username and Password not accepted

--- redis state after exhaustion ---
  bull:__probe_fail:1  [hash,   fields=12]
  bull:__probe_fail:failed [zset, card=1]     // ← NEW
  bull:__probe_fail:events [stream]

--- job record ---
```

```
failedReason : 535-5.7.8 Username and Password not accepted
processedOn  : 1790508373584
finishedOn   : 1790508373627
getState()   : failed
attemptsMade : 3
failed zset  : 1 1790508373627            // id + finishedOn as the score
```

The terminal state after exhaustion mirrors a real dead-letter queue: the job ID becomes a member of the `failed` sorted set, scored by its finish time, and `failedReason` retains the last error verbatim.

> **This is the exact shape of the 11 stuck jobs in the live email queue**
> Reading `bull:email:failed` during setup returned the same structure, each entry carrying the Gmail rejection as `failedReason`. That single field is what identified the problem as an SMTP credential fault rather than a code defect — the application logs and the HTTP responses were all silent about it.

A measurement caveat worth recording: in the raw Redis hash, `attemptsMade` was *absent* even though the job had been attempted three times; the typed `job.attemptsMade` getter correctly reported `3`. Attempt bookkeeping is not stored under that plain field name. Always read it through the API rather than from `hgetall`, or you will conclude a retried job was never attempted.

11 Both workers attach observers so failures are visible in the worker console:

```
worker.on("completed", (job) => console.log(`[email-worker] job ${job.id} … completed`));
worker.on("failed", (job, error) => console.error(`[email-worker] job ${job?.id} … failed:`, error));
```

`worker.ts:28,34` and `cron-worker.ts:42,48`

Note the asymmetry in retention: successes are removed immediately, but `removeOnFail: 1000` keeps failures. That is the correct default for this workload — a silently dropped failure is far worse than a retained one — and it is precisely why the queue was able to tell us what had gone wrong after the fact.

PHASE 8 Graceful shutdown

12 `src/queue/worker.ts:41-48`

```
const shutdown = async () => {
  console.log("[email-worker] shutting down...");
  await worker.close();
  process.exit(0);
};
process.on("SIGTERM", shutdown);
process.on("SIGINT", shutdown);
```

`worker.close()` **drains rather than aborts**: it stops claiming new jobs and waits for in-flight processors to finish before resolving. Combined with the stalled checker, a job interrupted mid-flight by a restart is re-queued and retried instead of vanishing — which is what you want, but it does mean a redeploy can re-send an email that had already been delivered.

Findings and risks

1. No audit trail for delivered email

`removeOnComplete: true` erases the record on success. Combined with the fact that the producer cannot observe the outcome, there is no way to answer "did user X receive their code?" from application data — the only trace is the recipient's inbox. Consider `removeOnComplete: { age: 3600, count: 500 }` to retain a short window for support investigations at negligible cost.

2. Retries can produce duplicate emails

Delivery semantics are at-least-once. If SMTP accepts a message but the connection drops before the worker sees the response, the job is retried and the user receives the code twice. With a 6-digit code and a 60-second resend cooldown this is survivable, but it is a real possibility. BullMQ's native `deduplication` option does not help here — it collapses enqueues, not retries of one job. A pragmatic guard is recording the OTP identifier against the job and skipping if already sent.

3. Redis eviction can silently destroy jobs

> **Upstash reports** `Eviction policy is optimistic-volatile. It should be "noeviction"`
>
> Under memory pressure Redis may evict keys. If a `bull:<queue>:<id>` job hash is evicted while its ID still sits in `wait`, the worker claims an ID with no backing record and the job is lost with no error surfaced anywhere. This did not occur during setup, but it is a structural risk for a queue on an evicting cache-tier database. Moving the queue to a dedicated Redis instance with `noeviction` removes the class of problem entirely.

4. A dead worker is indistinguishable from an idle queue

Nothing surfaces the condition where the worker exits cleanly but the backlog grows. The `/api/health` route returns `{ status: "ok" }` unconditionally and does not inspect queue depth, so a monitoring check would report a healthy system while OTPs silently accumulate. A worthwhile addition is exposing `getJobCounts()` for `wait` and `failed` and alerting on a non-zero failed count.

5. Cron correctness spans three services

As noted in Phase 4, a scheduled trigger traverses the cron worker, Inngest, and the registered function. Because `cron-worker.ts:48` only logs on `failed`, an Inngest-side rejection surfaces as a misleading queue failure. Worth confirming all three are running before debugging a cron trigger.

6. Producer-side degradation differs by design

Producers degrade to inline sending; workers refuse to start. Correct as written, but it means the *same* missing env var produces two very different symptoms — slow requests in one case, no emails at all in the other. Both log clearly, which is the mitigation; the asymmetry is intentional and should be preserved.

Operational reference

QUESTION	WHERE TO LOOK
How many jobs are waiting?	`LLEN bull:email:wait`
What is stuck right now?	`LRANGE bull:email:active 0 -1`
What failed, and why?	`ZRANGE bull:email:failed 0 -1` → `HGETALL bull:email:<id>` → read `failedReason`
Full state of one job	`HGETALL bull:email:<id>`
Is a worker alive?	`EXISTS bull:email:stalled-check`
What command is running	`npm run worker` / `npm run cron-worker`
Both, side by side	`npm run dev:all` (mprocs)

Method note

Every Redis state shown in phases 3, 5, 6 and 7 was captured live from a throwaway probe queue on the project's own Upstash instance, with a processor deliberately blocked so the in-flight state was observable. Both probe queues were removed afterwards and neither touched `bull:email` or `bull:cron`. Nothing in this document is inferred from BullMQ's published behaviour alone.

— end of document —